The OMI Collapse

Program, File, Rewrite

Every mature computational doctrine collapses a distinction that earlier systems treated as fundamental.

Lisp collapses program and object.

POSIX collapses storage and communication.

OMI collapses representation and interpretation.

System	Collapse	Canonical artifact
Lisp	program ↔ object	S-expression
POSIX	storage ↔ communication	file descriptor
OMI	representation ↔ interpretation	rewrite surface

The OMI artifact is not a record, file, object, glyph, packet, or database row.

The OMI artifact is the **rewrite surface**: a bounded binary surface whose meaning is determined by the active interpretation.

The Core Distinction

Traditional systems say:

```
data changes
```

OMI says:

```
the source remains; the reading changes
```

The same binary surface can be read as:

```
text
address
instruction
proof
```

```
receipt
graph
nomogram
rewrite table
projection
```

The bits are not necessarily mutated.

The interpretation surface is rewritten.

That rewrite is computation.

Notation as Cipher

OMI's shortest doctrine is:

```
notation as cipher
cipher as notation
```

A notation is not merely a display convention.

A notation is a rule for reading a binary surface.

A cipher is not only a secrecy mechanism.

A cipher is a structural transformation of interpretation.

Therefore, in OMI:

```
notation = declared cipher surface
cipher = operational notation surface
```

The purpose is not to hide meaning.

The purpose is to declare a lawful reading.

Control Codes as Rewrite Operators

Traditional computing treats ASCII control codes as non-printing characters.

OMI treats them as rewrite operators.

```
0x00..0x1F
```

does not primarily mean “invisible symbols.”

It means:

```
start
stop
shift
escape
acknowledge
separate
return
advance
interrupt
declare
incorporate
invalidate
```

These are not glyphs.

They are interpretation transitions.

They are cipher operators.

ASCII and Unicode Reinterpreted

Traditional view:

```
ASCII = lookup table
Unicode = character set
```

OMI view:

```
ASCII = incidence geometry
Unicode = rewrite manifold
```

Rows, columns, planes, private-use regions, sentinels, surrogates, and control ranges are not only storage ranges.

They are possible interpretation boundaries.

A codepoint is not only a character.

A codepoint may be:

```
point
block
selector
boundary
sentinel
carrier
projection
closure witness
```

Thus character space becomes rewrite space.

The Arithmetic of Closure

OMI begins from finite wheels.

A 4-bit wheel has:

```
16 states
```

OMI reads this as:

```
15 active states + 1 closure state
```

This is the canonical:

```
15-of-16
```

A 3-bit wheel has:

```
8 states
```

OMI reads this as:

```
7 active states + 1 closure state
```

This is the canonical:

```
7-of-8
```

The missing state is not lost.

It is the projective center.

The closure state is both:

```
origin
container
reset
witness
```

It is the point where the wheel folds back into itself.

Rotation Preserves What Shift Discards

Traditional bit manipulation often uses shifts.

A shift discards information.

OMI uses rotations.

A rotation preserves information.

The state does not fall off the edge.

The state moves around the wheel.

The canonical motion law is:

```
ΔC(x) = rotl(x,1) ⊕ rotl(x,3) ⊕ rotr(x,2) ⊕ C
```

Here:

```
C = carried closure of the orbit
```

The closure variable is not just a constant.

It is the witness left by the previous rewrite.

Every orbit contributes to future interpretation.

Memory is accumulated closure.

Rewrite Registered at Address

Traditional memory says:

```
address → stored value
```

OMI says:

```
address → registered rewrite
```

An OMI address does not merely locate data.

It locates a lawful transformation of interpretation.

The object is not the value stored at an address.

The object is the receipted rewrite registered at that address.

Projection Is Not Authority

A projection may show:

```
glyph
card
graph
DOM node
CSS transform
barcode
emoji
canvas object
portal view
```

But projection is not authority.

The reader may recognize a projection.

The resolver may promote a projection.

Only receipt accepts lawful interpretation.

Therefore:

```
reader recognizes
resolver promotes
receipt accepts
```

This prevents rendered appearance from becoming false authority.

The OMI Collapse

Lisp discovered that program and object are the same surface under different readings.

POSIX discovered that storage and communication are the same surface under different handles.

OMI discovers that representation and interpretation are the same binary surface under different rewrite routes.

The same surface can be notation.

The same surface can be cipher.

The distinction is not in the bits.

The distinction is in the active reading.

One-Line Canon

Lisp collapsed program and object. POSIX collapsed storage and communication. OMI collapses representation and interpretation: the same binary surface is both notation and cipher, distinguished only by the active reading, because computation is the rewrite of interpretation, not the mutation of data.

Shortest Form

```
OMI is notation as cipher and cipher as notation.
```